

Esercizio 1 (Pekko Cluster)

- Introduzione

In questo esercizio viene estesa l'implementazione centralizzata dell'allarme introducendo un'architettura distribuita basata su Apache Pekko Cluster Sharding. L'obiettivo è separare logicamente i componenti del sistema (controller, sensori, keypad) distribuendoli su nodi diversi del cluster, mantenendo però la stessa logica funzionale del caso centralizzato.

- Differenze principali rispetto alla versione centralizzata

Nel caso centralizzato esisteva un unico ActorSystem, ora nella versione distribuita vengono avviati 3 nodi distinti (sulla stessa JVM per semplicità): un nodo che ospita l'entità sharded del controller, un nodo per i sensori e un nodo per il keypad.

La differenza più importante è che il controller non è più un attore locale, ma un'entità gestita dal Cluster Sharding, ora infatti:

- viene registrato tramite Entity(TypeKey)(...)
- viene creato dal cluster quando riceve il primo messaggio
- può essere ricreato automaticamente dopo failure

I sensori e il keypad ricevevano un ActorRef[Command] diretto al controller, nel caso distribuito non è più possibile fare ciò, è necessario sfruttare l'EntityRef del controller che è un riferimento logico ad esso, questo riferimento è molto adeguato al caso distribuito, infatti:

- è indipendente dal nodo su cui gira l'entità
- resiliente a restart derivante da eventuali failure
- sempre valido anche se il controller viene ricreato

- Recovery Mode

L'unica differenza "funzionale" aggiunta rispetto al caso concentrato è questo nuovo stato di recovery che serve proprio per rendere il sistema resistente ad eventuali failure (che nel distribuito non sono così rari).

Questo stato di recovery è il nuovo stato di default quando il controller viene avviato (o restartato), il primo pin trasmesso dal keypad non fa altro che uscire da questo stato di recovery per portare il sistema in condizioni di lavoro "classiche" riportandolo allo stato "disarmed", da qui in poi il funzionamento è identico al caso concentrato.

- Conclusioni

Le modifiche introdotte rendono il sistema più robusto, scalabile e aderente a un'architettura realmente distribuita, pur mantenendo invariata la semantica dell'allarme.

Esercizio 2 (Java RMI)

- Analisi del problema

L'implementazione di un sistema di gioco distribuito come Tic-Tac-Toe presenta aspetti delicato per quanto riguarda la programmazione concorrente dei sistemi ad oggetti distribuiti:

- **Accesso concorrente alle risorse condivise:** Il server può gestire più istanze di gioco contemporaneamente attraverso una mappa (Map<String, Game>). Più client possono tentare di creare o unirsi a partite nello stesso momento, bisogna gestire correttamente le strutture dati del server.
- **Gestione dello stato del gioco:** Ogni partita possiede uno stato interno (la Board, la lista dei giocatori, la fase di attesa). Le transizioni di stato (es. il passaggio da "in attesa" a "in corso") devono essere atomiche per evitare race conditions, come per esempio due giocatori che si uniscono alla stessa partita contemporaneamente.
- **Comunicazione sincrona:** Java RMI utilizza chiamate remote sincrone, il che è comodo ma potrebbe nascondere problemi più complessi, per esempio il rischio di deadlock distribuito: se il server mantiene un lock su un oggetto Game mentre effettua una chiamata verso un client (GameListener), e quel client a sua volta tenta di chiamare un metodo del server che richiede lo stesso lock, il sistema si blocca.
- **Thread Pool in RMI:** Il framework RMI gestisce un pool di thread unbounded, creando un nuovo thread per ogni chiamata remota in arrivo. Questo permette a più partite di procedere in parallelo senza bloccarsi a vicenda, ma implica che con molte partite attive il numero di thread possa crescere significativamente. È fondamentale garantire che le modifiche alle variabili condivise siano protette da sincronizzazione esplicita.

- Strategia adottata

Ho utilizzato una specie di sincronizzazione a due livelli:

Livello Service: Utilizzo di metodi synchronized e blocchi dedicati per proteggere la mappa che associa il nome della partita con l'oggetto "Game" durante le operazioni di createGame, joinGame e listOpenGames.

Livello Game: Ogni istanza di gioco protegge i propri dati interni (board e flag) utilizzando blocchi di codice synchronized.

Per evitare il deadlock distribuito, il sistema segue la logica di rilasciare i lock prima di effettuare chiamate RMI verso i client.

- Nel metodo join, lo stato waiting viene aggiornato all'interno di un blocco protetto, ma le notifiche ai listener (receiveMessage, passTurn) vengono inviate dopo l'uscita dal blocco.

- Nel metodo `applyMove`, la modifica della Board è sincronizzata, ma tutte le comunicazioni di rete (notifica mossa, vittoria, cambio turno) avvengono al di fuori del blocco `synchronized`.

Atomicità del turno: La struttura del passaggio di turno è implementata come una catena di chiamate remote sincrone annidate: il server chiama `passTurn` sul client e rimane bloccato finché il metodo non ritorna. Il client, dentro `passTurn`, aspetta l'input dell'utente e poi chiama `makeMove` sul server, bloccandosi a sua volta. Il server esegue `applyMove` e al termine chiama `passTurn` sull'avversario, aggiungendo un ulteriore livello alla catena. Questo garantisce che solo il giocatore di turno possa modificare la board, ma implica che per ogni partita attiva rimangano occupati più thread del pool RMI simultaneamente, che se il sistema fosse reale non sarebbe ottimale.

Gestione asincrona dell'inizio partita: L'inizio di una partita viene gestito in modo asincrono: l'host che crea il match si mette in attesa che qualche joiner entri nel match. Nel momento in cui entra il joiner è lui che gestisce concretamente l'inizio, comunicando all'host di essere entrato e passano il turno (è un po' come se il joiner comunicasse "sono entrato nella partita, puoi effettuare la prima mossa") il flusso di gioco continua seguendo la logica che i due giocatori si passano il turno a vicenda una volta fatta la loro mossa. Il server monitora gli esiti della board (`checkWinner`, `isFull`) per notificare correttamente la fine della sessione e loggare l'evento.

- Conclusioni

Lo sviluppo del sistema ha permesso di verificare l'efficacia di Java RMI nella creazione di applicazioni distribuite basate su oggetti, ho apprezzato particolarmente la semplicità di questo framework per quanto si avvicini alla logica classica della oop, rendendo per chi scrive il codice "poco pesante" il fatto di lavorare nel distribuito.

Esercizio 3 (RabbitMQ)

- Analisi del problema

Il problema consiste nel garantire la **mutua esclusione** tra un insieme di processi indipendenti (*Worker*) che competono per l'accesso a una risorsa condivisa in un sistema distribuito.

Gli aspetti da garantire nella modellazione sono:

- **Safety:** In nessun istante di tempo più di un worker deve trovarsi all'interno della sezione critica e assenza di deadlock.
- **Assenza di memoria condivisa fra worker:** I processi visto che non sono sulla stessa JVM non possono comunicare tramite variabili comuni, il coordinamento deve avvenire tramite scambio di messaggi.

- **No starvation:** Il sistema deve garantire che tutti i worker che ne richiedono l'accesso possano prima o poi accedere alla risorsa condivisa.
- **Inizializzazione:** In un ambiente distribuito è necessario un meccanismo per stabilire lo stato iniziale del sistema senza generare conflitti (es. evitare la creazione di più permessi).

- **Strategia adottata**

La soluzione implementa un algoritmo basato su un token distribuito, utilizzando RabbitMQ come MOM.

- **Architettura “a Gettone”:** Il diritto di accesso alla sezione critica è rappresentato da un unico messaggio RabbitMQ chiamato "TOKEN" sulla coda creata (il che rimane incapsulato nel Middleware e nascosto al worker). Per entrare nella sezione critica, un worker deve semplicemente sfruttare l'API comodamente, senza preoccuparsi di niente, stessa cosa per uscire dalla sezione critica. La mutua esclusione è garantita dalla struttura interna del Middleware e dall'uso della BlockingQueue.

- **Sincronizzazione locale tramite BlockingQueue:** La deliverCallback di RabbitMQ viene eseguita da un thread interno della libreria, separato dal thread del Worker. La LinkedBlockingQueue di capacità 1 funge da ponte thread-safe tra i due: il thread RabbitMQ chiama put() quando il token arriva, mentre il thread del Worker si blocca su take() in enterCriticalSection. Il comportamento è analogo a quello di un semaforo mutex, ma il meccanismo di mutua esclusione reale è distribuito: è la coda RabbitMQ con il token circolante che coordina i worker.

- **Inizializzazione del sistema:** Per risolvere il problema del bootstrap, è stato introdotto il componente “SystemInit” che ha il compito di: dichiarare inizialmente la coda (mutex_token), effettuare la pulizia di eventuali esecuzioni precedenti (per esempio evitare che possano essere presenti 2 token nel sistema), iniettare l'unico token disponibile nel sistema (il che assicura che il sistema parta sempre da uno stato “sicuro”).

- **Incapsulamento:** Tutta la logica di connessione, dichiarazione e consumo è incapsulata nella classe Middleware, permettendo al Worker di concentrarsi solo sulla logica applicativa, accedendo alla sezione critica tramite un API chiara e semplice.

- Ogni worker è identificato dinamicamente tramite il proprio PID, potendo così runnare lo stesso file per creare molteplici worker, senza collisioni di nomi o necessità di inserirli a mano.

- **Conclusioni**

In conclusione, la combinazione tra istruzioni semplici di RabbitMQ e le strutture di sincronizzazione di Java offre una soluzione scalabile, robusta e facilmente implementabile per il coordinamento di processi.

